BACKEND DEVELOPER · ESERCIZIO 2

Images CDN

Un CDN distribuito on-premise che salva immagini e le serve via HTTP: il design del sistema, più un prototipo che lo dimostra.

Nome = impronta del contenuto | Copia su 2 nodi subito | Registro degli upload | Recupero automatico

Federico Pagnossin · settembre 2026

IL PROBLEMA

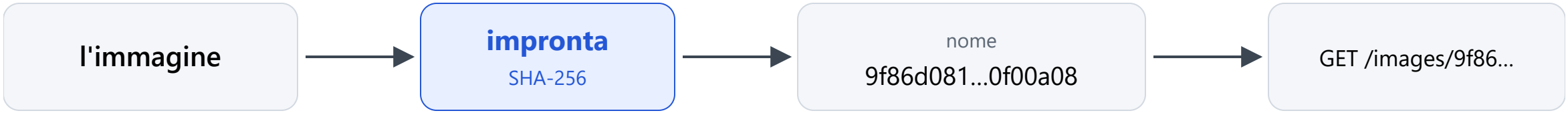
Tre requisiti

R1	**Un'immagine caricata deve arrivare su tutti i nodi, il prima possibile**	Ogni upload viene annunciato su un registro condiviso: ogni nodo lo legge e scarica l'immagine
R2	**Ogni nodo deve rispondere, anche se non ha ancora l'immagine**	Il nodo la chiede agli altri, la restituisce e se ne tiene una copia
R3	**Un nodo che si riavvia deve rimettersi in pari velocemente**	Il registro ricorda fin dove era arrivato: il nodo recupera solo quello che ha perso

Vincolo: le immagini non vengono mai cancellate. È proprio questo che semplifica tutto il resto.

L'IDEA CHIAVE

Il nome di un'immagine è l'impronta del suo contenuto



Stessa immagine, stesso nome. E un'immagine non cambia mai e non viene mai cancellata. Quindi:

Nessun conflitto
Due nodi non possono avere versioni diverse della stessa immagine.

Ripetere è sicuro
Caricare due volte, o ricevere due volte lo stesso annuncio, non ha effetti.

Integrità verificabile
Chi riceve un file ricalcola l'impronta: se non corrisponde, lo scarta.

Cache per sempre
Browser e CDN possono conservarla senza mai ricontrollarla.

ARCHITETTURA

Nodi tutti uguali e un registro condiviso



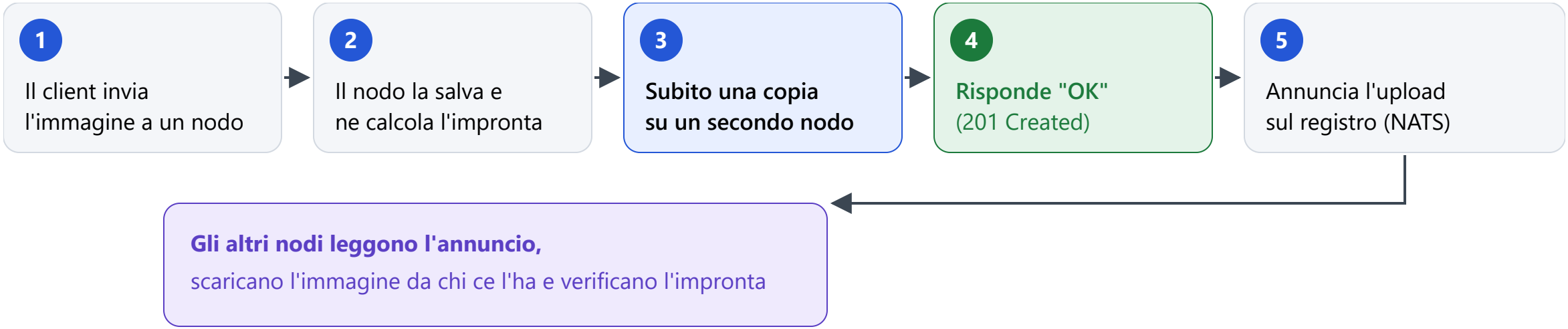
Nessun capo: se un nodo cade, gli altri continuano.

NATS registra gli upload: solo l'annuncio, non il file.

I file viaggiano direttamente da un nodo all'altro.

R1 · UPLOAD

Su 2 nodi subito, su tutti in un attimo



Perché la copia prima di rispondere?

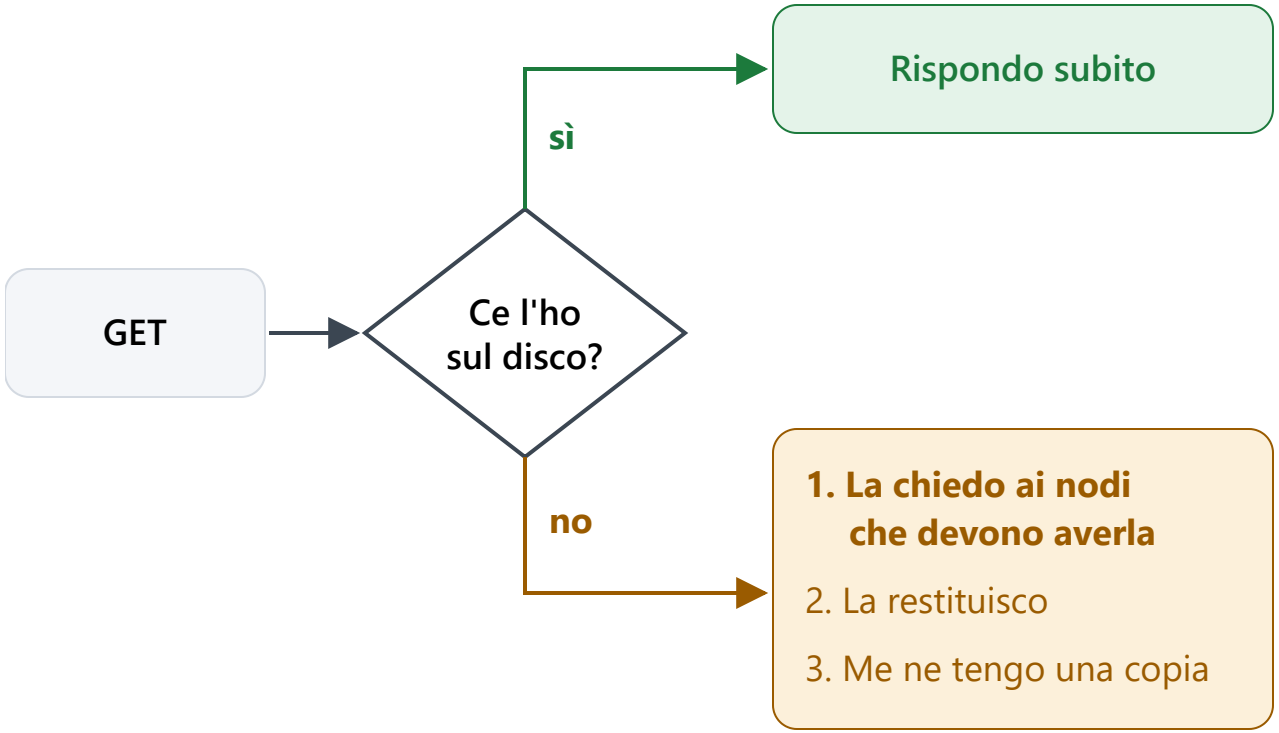
Se il primo nodo si rompe subito dopo, l'immagine non è persa: è già su un altro.

E se il registro non risponde?

L'upload riesce lo stesso: l'annuncio resta in coda sul disco e parte appena possibile.

Di solito l'immagine è su tutti i nodi in meno di un secondo

Ogni nodo risponde, sempre



Come sa a chi chiederla?

Ogni nodo calcola da solo quali nodi hanno ricevuto la copia (rendezvous hashing). Nessun servizio centrale da interrogare.

La volta dopo

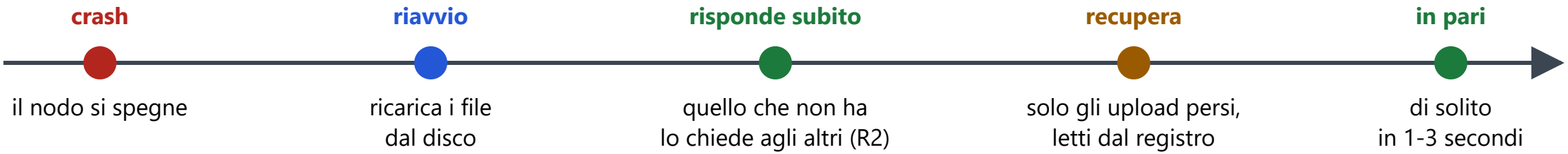
La copia tenuta la rende locale: la richiesta successiva è immediata.

Protezioni

Timeout di 2 secondi; un nodo che non risponde viene saltato; 100 richieste per la stessa immagine fanno un solo download.

R3 · RIAVVIO

Utile subito, in pari in pochi secondi



Perché è veloce

Il registro ricorda, per ogni nodo, l'ultimo upload che ha elaborato. Al riavvio il nodo riceve solo quelli successivi: niente confronti completi.

Rete di sicurezza

Ogni 30 secondi i nodi confrontano un riassunto di ciò che hanno e scaricano quello che manca (anti-entropy). Serve per i casi rari: nodo spento per giorni, disco sostituito.

ROBUSTEZZA

E se qualcosa va storto?

Un nodo si spegne	Il load balancer lo esclude; gli altri nodi hanno tutte le immagini.
Il registro (NATS) non risponde	Upload e letture funzionano lo stesso; gli annunci partono appena torna.
Un nodo è lento	Dopo 2 secondi si chiede a un altro; per qualche secondo non lo si interroga più.
Un file è corrotto	L'impronta non corrisponde: il file viene scartato e preso da un altro nodo.
Il disco si sta riempiendo	I nuovi upload vengono rifiutati prima che il disco sia pieno del tutto.
La rete si divide in due	Le due parti continuano a lavorare; poi si riallineano senza conflitti.

LE SCELTE

Cosa ho scelto, e cosa ho scambiato

Allineamento in pochi secondi, non istantaneo

Il testo chiede "il prima possibile". Aspettare tutti i nodi renderebbe gli upload lenti, e impossibili con un nodo giù. In cambio: 2 copie subito e lettura da qualsiasi nodo.

Un registro, non invii diretti

Con gli invii diretti, un nodo spento perderebbe gli upload. Il registro li conserva e ricorda fin dove è arrivato ogni nodo.

NATS JetStream

Un solo programma, leggero da gestire on-premise. Kafka farebbe lo stesso lavoro, se l'azienda lo usa già.

Ogni nodo ha tutte le immagini

Letture velocissime, ma lo spazio totale è quello di un solo disco. Se servisse di più: copie solo su alcuni nodi, con lo stesso meccanismo.

IL PROTOTIPO

Il design, funzionante

In locale

`docker compose up` : 3 nodi in container separati, NATS e una dashboard. Crash vero con `docker compose kill node3` .

Online

Una variante compatta in un solo container, per i limiti del piano gratuito.

Verificato

61 test automatici. A ogni modifica la CI ripete i tre scenari R1, R2, R3.

Semplificato rispetto al design

FastAPI al posto di Nginx per servire i file

Un solo server NATS invece di tre

Un piccolo gateway al posto di HAProxy

Immagini tenute in memoria (massimo 10 MB)

Il nome basato sul contenuto elimina i conflitti; la copia immediata e la lettura dagli altri nodi fanno rispondere sempre qualsiasi nodo; il registro rende il recupero preciso.